《最优化算法与理论》实验报告

实验序号:	2
实验内容:	下降算法比较
班　　级:	信息 2301
学　　号:	202310040128
姓　　名:	雷龙浩
完成日期:	2026 年 03 月 29 日

目录

1 算法原理与实现概要

1.1 下降算法的一般迭代格式

本次实验涉及的三类下降算法均遵循如下迭代形式：

$$x_{k+1} = x_k + \alpha_k d_k, \tag{1}$$

其中 d_k 为搜索方向，$\alpha_k > 0$ 由线搜索确定。

1.2 最速下降法（Steepest Descent）

最速下降法取负梯度方向作为搜索方向：

$$d_k = -\nabla f(x_k). \tag{2}$$

该方法实现简单、每步计算量小，但当目标函数等高线高度扭曲时，容易在谷底两侧来回摆动，从而出现典型的锯齿现象，整体收敛速度通常较慢。

1.3 牛顿法（Newton Method）

牛顿法利用二阶信息构造方向：

$$d_k = -[\nabla^2 f(x_k)]^{-1} \nabla f(x_k), \tag{3}$$

实现中通常以求解线性方程组 $\nabla^2 f(x_k) d_k = -\nabla f(x_k)$ 的方式获得 d_k。当 Hessian 正定且当前位置已经接近极小点时，牛顿法通常具有很快的局部收敛速度，因此在中小规模光滑问题中表现往往明显优于一阶方法。

1.4 修正牛顿法（Modified Newton）与修正 Cholesky

当 Hessian 非正定时，直接使用牛顿方向可能失去下降性，线搜索也可能无法接受该方向。为此，修正牛顿法不直接求解

$$\nabla^2 f(x_k) d_k = -\nabla f(x_k),$$

而是先构造一个正定近似矩阵 B_k，再求解

$$B_k d_k = -\nabla f(x_k). \tag{4}$$

本实验中的修正牛顿法采用课件中的修正 Cholesky 思想，将矩阵分解为

$$B_k = LDL^T.$$

其中每一列分解时，对角元按照

$$d_j = \max\left\{ |c_{jj}|, \left(\frac{\theta_j}{\beta}\right)^2, \delta \right\}$$

进行修正，从而保证 $d_j > 0$，最终保证 B_k 为正定矩阵。这样做的直接效果是：即使原始 Hessian 含有负曲率信息，算法仍然可以构造出可靠的下降方向。

2 实验设置

2.1 代码结构与运行方式

本次实验的代码按模块化方式组织：

- `optimization/core.py`：目标函数、梯度、Hessian 的统一封装；
- `optimization/line_search/`：黄金分割、Armijo、Wolfe-Powell 三种线搜索；
- `optimization/optimizers/`：最速下降法、牛顿法、修正牛顿法；
- `work2/work2.py`：实验入口、结果打印与图片绘制。

在项目根目录运行

```
python work2/work2.py
```

即可完成四组实验，并生成报告所需图片。

2.2 主要参数设置

表 1: 实验中的主要参数设置

参数名称	符号	取值	说明
Rosenbrock/Rastrigin 停止阈值	ϵ	10^{-5}	当 $\|\nabla f(x_k)\| \le \epsilon$ 时停止
a9a 二阶法停止阈值	ϵ	10^{-6}	为了更清楚比较二阶法精度表现
修正 Cholesky 下界	δ	按课件公式自动计算	代码中取 $\delta = \epsilon_{\text{mach}} \max(\gamma + \xi, 1)$
逻辑回归正则项	λ	$1/(100m)$	m 为样本数，防止过拟合

2.3 线搜索参数设置

表 2: 线搜索参数

线搜索方法	参数配置	说明
Armijo	$\alpha_0 = 1,\ \rho = 0.5,\ c_1 = 10^{-4}$	回溯收缩，保证充分下降
Wolfe-Powell	$\alpha = 1,\ \beta = 0.5,\ c_1 = 10^{-4},\ c_2 = 0.9$	同时满足充分下降与曲率条件
黄金分割	$[a_k, b_k] = [0, 2],\ \text{tol} = 10^{-4}$	一维精确线搜索

2.4 关键代码展示

由于完整代码较长，正文中只展示与本次实验最相关的两段核心实现。

1. a9a 逻辑回归目标函数、梯度与 Hessian 定义

Listing 1: work2.py 中逻辑回归目标函数的关键实现

```python
def logistic_objective_from_csv(csv_path: Path) -> Objective:
    """
    a9a 逻辑回归目标函数:
        (1/m) * sum(log(1 + exp(-b_i * a_i^T x))) + lambda||x||^2
        lambda = 1 / (100m)
    CSV 约定: 第一列为标签 b_i, 其余列为特征 a_i。

    与作业 PDF 的记号对应关系:
        - 代码里的 y    <-> 公式里的 b
        - 代码里的 xmat <-> 公式里的 A
        - 代码里的 w    <-> 公式里的 x
    因此 y * (xmat @ w) 就是在批量计算每个样本的 b_i a_i^T x。

    返回:
        Objective(func, grad, hess)
    """

    # 读入数据并拆成标签/特征矩阵。
    # 这里默认标签已经编码成 {-1, 1}, 与作业里的 b_i 写法一致。
    data = np.loadtxt(csv_path, delimiter=",", dtype=float)
    y = data[:, 0]
    xmat = data[:, 1:]
    m, n = xmat.shape
    # 与作业一致的 L2 正则系数, m 越大正则越弱。
    lam = 1.0 / (100.0 * m)
    eye = np.eye(n, dtype=float)

    def func(w: np.ndarray) -> float:
        # yz[i] = b_i * a_i^T x
        yz = y * (xmat @ w)
        # logaddexp(0, -yz) 等价于 log(1 + exp(-b_i a_i^T x)),
        return float(np.logaddexp(0.0, -yz).mean() + lam * np.dot(w, w))

    def grad(w: np.ndarray) -> np.ndarray:
        # yz = b_i * a_i^T x, 后面公式更紧凑。
        yz = y * (xmat @ w)
        # prob[i] = 1 / (1 + exp(b_i a_i^T x))
        # 也就是 sigmoid(-b_i a_i^T x)。
        prob = 1.0 / (1.0 + np.exp(yz))
```

```
        # 梯度公式:
        #   -(1/m) * sum(b_i a_i * sigmoid(-b_i a_i^T x)) + 2 * lambda * x
        # 写成矩阵形式后就是下面这一行。
        return -(xmat.T @ (y * prob)) / m + 2.0 * lam * w

    def hess(w: np.ndarray) -> np.ndarray:
        yz = y * (xmat @ w)
        prob = 1.0 / (1.0 + np.exp(yz))
        # weight[i] = sigmoid(-b_i a_i^T x) * (1 - sigmoid(-b_i a_i^T x))
        # 对应 Hessian 中对角权重矩阵 W 的第 i 个对角元。
        weight = prob * (1.0 - prob)
        # Hessian = A^T W A / m + 2 * lambda * I
        # 这里没有显式构造巨大的对角矩阵 W，而是利用广播把每一行样本
        # 按 weight[i] 缩放后，再做矩阵乘法，结果完全等价。
```

2. 修正牛顿法中的修正 Cholesky 核心步骤

Listing 2: modified_newton.py 中修正 Cholesky 的关键实现

```
def _modified_cholesky(a: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
    """
    修正 Cholesky 分解（课件 Algorithm 5），用于把 Hessian 修成“可分解且正定”的形式。

    目标：给定对称矩阵 A，构造正定矩阵 B，并给出分解：
        B = L D L^T
    其中 L 为单位下三角（对角为 1），D = diag(d1,...,dn) 且 di > 0。

    关键修正点（保证正定）：在第 j 步，先算
        theta_j = max_{i>j} |c_ij|
    再选
        d_j = max(|c_jj|, (theta_j/beta)^2, delta)
    并用
        c_ii <- c_ii - c_ij^2 / d_j
    更新后续对角，从而保证所有 d_j 都有正的下界。

    参数（按课件符号）：
    - epsilon: 机器精度（float64 下约为 2.22e-16）
    - gamma(A) = max_i |a_ii|
    - xi(A) = max_{i!=j} |a_ij|
```

```
    - delta = epsilon * max(gamma(A) + xi(A), 1)
    - beta = sqrt(max(gamma(A), xi(A)/sqrt(n^2-1), epsilon))

    返回:
    - L: 单位下三角矩阵（对角为 1）
    - d: 对角向量（D = diag(d)，且 d_i > 0）
    """

    a = np.asarray(a, dtype=float)
    assert a.ndim == 2 and a.shape[0] == a.shape[1], "A must be a square matrix."

    n = a.shape[0]
    if n == 0:
        return np.eye(0, dtype=float), np.zeros((0,), dtype=float)

    eps = float(np.finfo(float).eps)  # 机器精度 epsilon
    gamma = float(np.max(np.abs(np.diag(a))))  # gamma(A) = max_i |a_ii|
    if n == 1:
        xi = 0.0
    else:
        off = a - np.diag(np.diag(a))
        xi = float(np.max(np.abs(off)))  # xi(A) = max_{i!=j} |a_ij|

    # delta = epsilon * max(gamma(A) + xi(A), 1)，用于给 d_j 一个正的下界。
    delta = eps * max(gamma + xi, 1.0)
    # beta = sqrt(max(gamma(A), xi(A)/sqrt(n^2-1), epsilon))，控制非对角项的影响。
    denom = float(np.sqrt(n * n - 1.0)) if n > 1 else 1.0
    beta = float(np.sqrt(max(gamma, xi / denom, eps)))

    c = a.copy()
    l = np.eye(n, dtype=float)
    d = np.zeros((n,), dtype=float)

    for j in range(n):
        # 下面的 j 对应课件里的第 (j+1) 步（课件是 1-based，这里是 0-based）。

```

```
        # 计算 L 的第 j 行: l_js = c_js / d_s,  s < j
        for s in range(j):
            l[j, s] = c[j, s] / d[s]

        # 计算消元后的列元素: c_ij = a_ij - sum_{s<j} l_js * c_is,  i > j
        for i in range(j + 1, n):
            if j == 0:
                c[i, j] = a[i, j]
            else:
                c[i, j] = a[i, j] - float(np.dot(l[j, :j], c[i, :j]))

        # theta_j = max_{i>j} |c_ij|, 刻画当前列非对角元素的最大幅度。
        theta = float(np.max(np.abs(c[j + 1 :, j]))) if j < n - 1 else 0.0
        # d_j = max(|c_jj|, (theta_j/beta)^2, delta), 保证 d_j >= delta > 0。
        d[j] = max(abs(float(c[j, j])), (theta / beta) ** 2, delta)

        # 更新后续对角: c_ii <- c_ii - c_ij^2/d_j, 相当于做一次 LDL^T 消元。
        for i in range(j + 1, n):
            c[i, i] = c[i, i] - (c[i, j] ** 2) / d[j]

    return l, d
```

3 实验结果分析与讨论

3.1 Rosenbrock 函数最小化

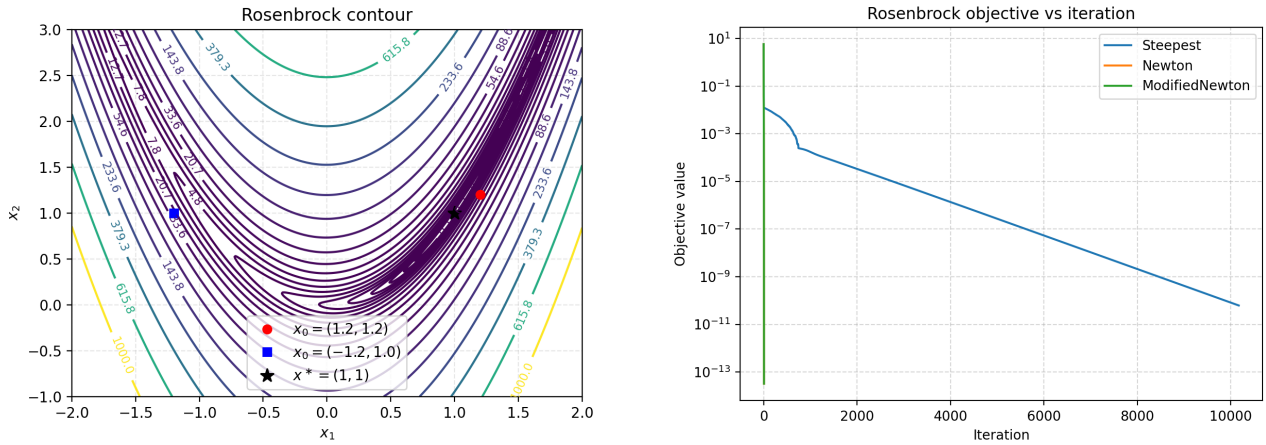
函数形式：

$$f(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2. \tag{5}$$

问题特征： Rosenbrock 函数存在一条狭长弯曲的香蕉形谷底，优化路径如果不能及时感知曲率变化，就会在谷底附近反复摆动，因此它非常适合用来比较一阶方法和二阶方法的差异。

表 3: Rosenbrock（$x_0 = (1.2, 1.2)$）三种下降法结果汇总

方法	迭代次数	f^*	是否收敛	$\|\nabla f(x^*)\|$
Steepest Descent	10171	6.1958×10^{-11}	是	9.87×10^{-6}
Newton Method	7	3.2267×10^{-14}	是	1.45×10^{-6}
Modified Newton	7	3.2267×10^{-14}	是	1.45×10^{-6}



(a) Rosenbrock 等高线图

(b) 目标函数值收敛曲线

图 1: Rosenbrock 问题的函数形状与收敛过程。

结合上表和上图可以看出，最速下降法虽然最终也能收敛到极小点附近，但需要超过一万次迭代；与之相比，牛顿法和修正牛顿法都只用了 7 次迭代便达到较高精度，说明二阶曲率信息在该类病态问题中非常关键。另外，在这个起点下牛顿法与修正牛顿法的轨迹完全一致，说明沿本次迭代路径 Hessian 的性质已经足够好，修正步骤没有显著改变方向。

3.2 线搜索对迭代次数的影响（Rosenbrock，x0=(-1.2,1)）

为了进一步观察步长策略的影响，固定初始点 $x_0 = (-1.2, 1)$，分别在三种下降法上使用黄金分割、Armijo 与 Wolfe-Powell 线搜索，结果见下表。

表 4: Rosenbrock（$x_0 = (-1.2, 1)$）线搜索对比结果

方法	线搜索	迭代次数	f^*	是否收敛
Steepest Descent	Golden Ratio	2409	1.2028×10^{-10}	是
Steepest Descent	Armijo	10916	6.1319×10^{-11}	是
Steepest Descent	Wolfe-Powell	10463	6.2082×10^{-11}	是
Newton Method	Golden Ratio	15	1.1657×10^{-16}	是
Newton Method	Armijo	21	3.7440×10^{-21}	是
Newton Method	Wolfe-Powell	21	3.7440×10^{-21}	是
Modified Newton	Golden Ratio	15	1.1650×10^{-16}	是
Modified Newton	Armijo	21	3.7440×10^{-21}	是
Modified Newton	Wolfe-Powell	21	3.7440×10^{-21}	是

可以看到，线搜索策略对最速下降法的影响尤其显著：在本例中，黄金分割法把迭代次数从约一万步降低到约两千四百步，效果非常明显。对牛顿法与修正牛顿法而言，不同线搜索也会影响迭代次数，但影响幅度相对较小，在这个二维光滑问题上，精确线搜索实现了更少的迭代次数。

3.3 Rastrigin 函数与初始点敏感性

函数形式：

$$f(x) = 10n + \sum_{i=1}^{n} \left(x_i^2 - 10\cos(2\pi x_i) \right). \tag{6}$$

问题特征： Rastrigin 函数具有强烈的周期振荡结构，局部极小点非常多，算法结果对起点和步长都比较敏感。下图给出了二维情形下的函数形状示意，虽然实际实验使用的是 $n = 6$，但二维图足以反映其多峰特征。

表 5: Rastrigin（$n = 6$）不同初始点下三种下降法结果

初始点编号	方法	迭代次数	f^*	是否收敛
#1	Steepest Descent	1	0.0000	是
#1	Newton Method	2	5.3728×10^1	是
#1	Modified Newton	2	5.3728×10^1	是
#2	Steepest Descent	1	0.0000	是
#2	Newton Method	2	5.3728×10^1	是
#2	Modified Newton	2	5.3728×10^1	是
#3	Steepest Descent	1	0.0000	是
#3	Newton Method	1	0.0000	是
#3	Modified Newton	9	2.7859×10^1	是

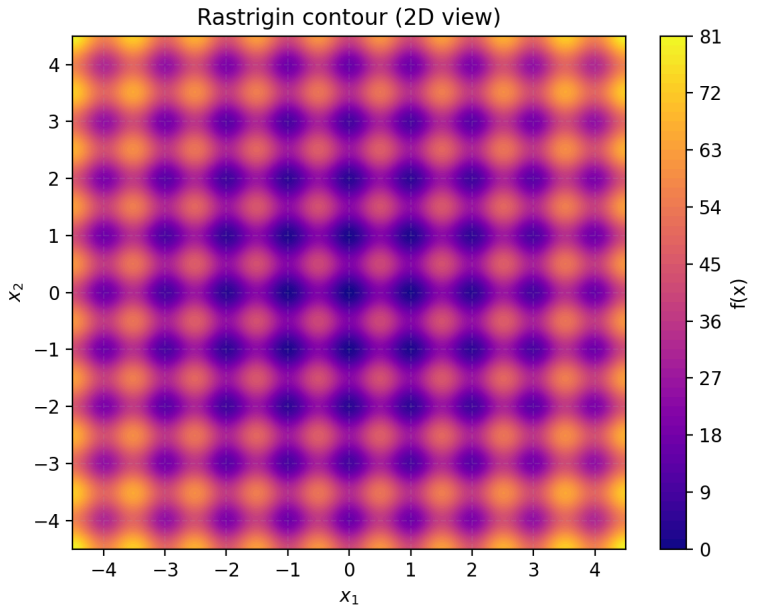


图 2: 二维 Rastrigin 函数的等高线图（实验中实际使用 $n = 6$，此图仅用于展示多峰结构）。

从上表可以看到，不同算法和不同初始点的结果差异很大，这体现了非凸多峰问题的难度。初始点 #1 和 #2 为例，最速下降法一步就到达了全局最优点附近，而牛顿法和修正牛顿法却停在函数值约为 53.728 的局部极小点；初始点 #3 下，牛顿法得到全局最优，而修正牛顿法则停在函数值约为 27.859 的另一个局部极小点。这说明对于非凸问题，**收敛得快**与**收敛到全局最优**是两件不同的事情，牛顿类方法虽然局部收敛很快，但并不自动意味着全局结果更优。

3.4 逻辑回归（a9a 数据集）性能对比

目标函数：

$$\min_x \frac{1}{m}\sum_{i=1}^{m}\log\left(1+\exp(-b_i a_i^T x)\right)+\lambda\|x\|_2^2. \tag{7}$$

问题特征： a9a 逻辑回归是一个典型的高维光滑优化问题。由于加入了 L_2 正则项，目标函数具有较好的数值稳定性，也比较适合观察二阶方法在实际数据集上的表现。

表 6: a9a 逻辑回归三种下降法结果

方法	迭代次数	f^*	是否收敛	$\|\nabla f(x^*)\|$
Steepest Descent	2000	3.2295×10^{-1}	否	1.906×10^{-4}
Newton Method	7	3.2266×10^{-1}	是	2.345×10^{-7}
Modified Newton	7	3.2266×10^{-1}	是	2.345×10^{-7}

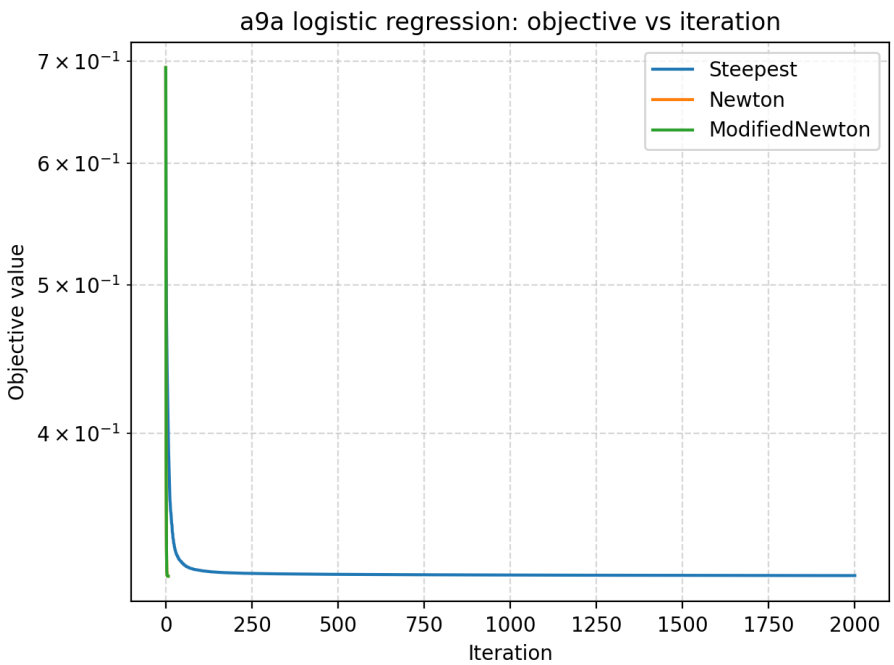


图 3: a9a 逻辑回归：目标函数值随迭代次数变化（半对数坐标）。

结合上表和收敛曲线可以看出，最速下降法在达到 2000 次迭代上限后仍未满足停止条件，而牛顿法与修正牛顿法都只用了 7 步便收敛。这里两种牛顿类方法结果完全一致，是因为逻辑回归加上二次正则后 Hessian 本身就是正定的，修正步骤没有触发。